

Ejercicio 2 – Arreglos de objetos

Editor de Filtros de Imágenes

Análisis:

1. ¿Qué clases son necesarias para resolver el problema? ¿Qué propiedades y métodos tendrá cada una de ellas?

Pixel modela un punto de la imagen con los atributos r, g y b, además de getters, setters con validación y getBrightness() para calcular el promedio del píxel. Image representa la matriz de píxeles mediante pixels y contiene métodos para obtener dimensiones, acceder y modificar píxeles, calcular el brillo promedio, encontrar el píxel más claro y oscuro y clonar la imagen. ImageEditor realiza las transformaciones y filtros como escala de grises, negativo, canales, brillo, blanco y negro, espejo horizontal y rotación de 90°. ImageUtils se encarga de leer y guardar archivos mediante load() y save(). Finalmente, Main o EditorImagenes contiene el método main(String[] args) y coordina la interfaz, las validaciones y las diferentes funciones del programa.

2. ¿Qué tipo deben tener las propiedades y métodos de cada clase?

Pixel utiliza los atributos r, g y b de tipo int, mientras que sus métodos accesorios y mutadores retornan int y void, respectivamente. Image utiliza pixels de tipo Pixel[[]], getHeight() y getWidth() retornan int, getPixel() retorna un Pixel y calculateAverageBrightness() retorna un double. ImageEditor contiene filtros que devuelven un nuevo objeto de tipo Image. Finalmente, ImageUtils utiliza load() para retornar un objeto Image y save() para guardar la imagen sin retornar ningún valor (void).

3. ¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo? ¿Cuántas dimensiones tendrá, qué representa cada una de ellas y qué tipo de dato almacenará en cada posición?

La propiedad pixels pertenece a la clase Image y tiene dos dimensiones: la primera representa la fila o eje Y (alto) y la segunda representa la columna o eje X (ancho), almacenando referencias a objetos de tipo Pixel.

4. ¿Cuáles deben ser los modificadores de visibilidad de los miembros de cada clase?

Todos los atributos deben ser private para respetar el principio de encapsulamiento, mientras que todos los métodos y constructores de las clases públicas deben ser public.

5. ¿Qué parámetros serán requeridos por los métodos en sus clases?

- `Pixel(int r, int g, int b)`
- `Image(int height, int width)` o `Image(Pixel[][] pixels)`
- `setPixel(int row, int col, Pixel p)`
- `brightness(int amount)`
- `blackAndWhite(int limit)`
- `keepOnlyChannel(int channel)`
- `ImageUtils.load(String filename)`
- `ImageUtils.save(Image image, String filename)`

6. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?

`Pixel` se crea mediante su constructor, pasando explícitamente los valores `r`, `g` y `b`, mientras que `Image` se crea instanciando la matriz `new Pixel[height][width]` según las dimensiones obtenidas al leer el archivo mediante `ImageUtils.load()`.

7. ¿De qué forma recibe el método `main` los argumentos de la línea de comandos? ¿Qué tipo de dato tienen y qué conversiones serán necesarias para poder utilizarlos?

Los parámetros se reciben mediante `String[] args` y son de tipo `String`; cuando se necesitan valores numéricos, tanto obligatorios como opcionales, deben convertirse a `int` utilizando `Integer.parseInt(args[i])`.

8. ¿Qué validaciones deben realizarse sobre los argumentos antes de intentar abrir la imagen?

Se deben realizar validaciones como `args.length >= 3`, comprobar que `!args[0].equals(args[1])` para evitar sobrescribir archivos, verificar la existencia del archivo mediante `new File(args[0]).exists()` y validar el nombre del filtro. Si alguna validación falla, se debe mostrar un mensaje de error en la consola, indicar la sintaxis correcta del comando y finalizar la ejecución utilizando `return`.

9. ¿Cómo determinara las dimensiones del arreglo si estas dependen del archivo que reciba el programa?

Primero se leen las dimensiones del archivo de imagen original utilizando `BufferedImage.getWidth()` y `getHeight()`, y luego se crea dinámicamente el arreglo de píxeles mediante `new Pixel[height][width]`.

10. ¿En qué momento se crean los objetos que se almacenan dentro del arreglo?
¿Qué contiene una posición del arreglo antes de que se le asigne un objeto?

Los objetos Pixel se crean iterativamente dentro del bucle de carga y procesamiento mediante `new Pixel(...)`; antes de asignarlos, cada posición del arreglo contiene una referencia nula (`null`).

11. ¿Cómo recorrerá todas las posiciones del arreglo para acceder a los atributos de cada uno de los objetos que contiene?

Mediante estructuras repetitivas anidadas, utilizando ciclos `for` anidados para recorrer y procesar los elementos.

12. ¿Qué diferencia existe entre asignar a una posición del arreglo la referencia de un objeto que ya existe y crear un objeto nuevo con los mismos valores? ¿En cuáles de las operaciones solicitadas es indispensable crear objetos nuevos?

Asignar la misma referencia hace que dos posiciones apunten al mismo espacio en memoria, por lo que un cambio realizado en una también afecta a la otra. En cambio, crear un nuevo objeto genera una instancia independiente. Por ello, es indispensable crear objetos nuevos al aplicar cualquier filtro y al clonar la imagen original para poder comparar correctamente el promedio de brillo.

13. ¿Qué debe ocurrir con las propiedades de ancho y alto cuando se ejecuta una operación que cambia el tamaño de la imagen?

Al realizar una rotación de 90° , la nueva imagen intercambia sus dimensiones: el nuevo alto corresponde al ancho original (`newHeight = oldWidth`) y el nuevo ancho corresponde al alto original (`newWidth = oldHeight`).

14. ¿Cómo evitará que los atributos de un píxel quedan fuera del rango permitido?

Se utiliza una técnica de truncamiento (*clamping*) dentro de los setters o en la lógica de cálculo para mantener los valores dentro de los límites permitidos.

Diseño:

